

VEI NEXUS

Learning Management System

A Full-Stack LMS for Academic Institution

Submitted By

Student ID	Name	Contribution
24K-0961	Mahad Khan	Frontend & Integration
24K-0884	Shahbaz Hassan	Backend & API
24K-1039	Samad Siddique	Database & Schema

Course: Database Systems (DBS)
Spring Semester 2026

1. Project Title

VEI NEXUS — Learning Management System (LMS)

VEI NEXUS is an institutional Learning Management System designed and developed as a full-stack web application that bridges students and teachers through a centralized digital platform.

2. Group Members

Student ID	Full Name	Role / Contribution
24K-0961	Mahad Khan	Frontend Development & System Integration
24K-0884	Shahbaz Hassan	Backend Development & REST API
24K-1039	Samad Siddique	Database Design & Schema Normalization

3. Project Description

3.1 Overview

VEI NEXUS is a web-based Learning Management System developed specifically for academic institutions. The system provides a unified digital environment where students and teachers can interact through structured academic workflows. The platform automates routine academic operations — including student registration, attendance tracking, and grade management — reducing administrative overhead and improving transparency for all stakeholders.

3.2 Technology Stack

Layer	Technology	Purpose
Frontend	HTML, CSS, JavaScript	User interface and client-side logic
Backend	Node.js (Express.js)	RESTful API and server-side processing
Database	Supabase (PostgreSQL)	Relational database and real-time data layer
Integration	Full-Stack (Frontend-BackendDB)	All layers fully integrated end-to-end

3.3 System Modules

Student Module

- View grades and marks submitted by teachers
- Self-registration using a pre-allocated student ID
- Login with institute-allocated credentials (ID and password)
- View personal academic dashboard
- Monitor attendance records across all enrolled courses

Teacher Module

- Secure teacher login via credential-based authentication
- Mark and manage student attendance for each course session
- Enter, update, and manage student grades and marks
- View class roster and enrolled student list
- Monitor overall class performance at a glance

Admin / System Module

- Institute-level management of users, courses, and enrollments
- Allocation of student and teacher IDs and credentials
- Course creation and teacher-course assignment

3.4 Key Features

- Role-based access control: separate portals for students and teachers
- Real-time data synchronization through Supabase's PostgreSQL backend
- Fully integrated frontend-backend-database architecture
- Attendance tracking with Present / Absent / Leave status per session

4. Project Requirements

4.1 Functional Requirements

FR-01: User Authentication

- The system shall allow students to register using an institute-allocated student ID
- The system shall authenticate users (students and teachers) via email/ID and password
- The system shall enforce role-based access — students cannot access teacher features and vice versa

FR-02: Student Features

Students shall be able to view their attendance records per course
Students shall be able to view their grades and marks per course
Students shall be able to view their enrolled courses

FR-03: Teacher Features

- Teachers shall be able to mark attendance for students on a per-session basis
- Teachers shall be able to enter and update grades for enrolled students
- Teachers shall be able to view the list of students enrolled in their courses

FR-04: Course Management

- The system shall support course creation with a title, code, and assigned teacher
- The system shall support student enrollment into courses
- Each course shall maintain its own attendance and grade records independently

FR-05: Data Integrity

- All foreign key relationships shall be enforced at the database level
- Duplicate enrollment of the same student in the same course shall be prevented • Attendance records shall be unique per student per course per date

4.2 Non-Functional Requirements

ID	Category	Requirement
NFR-01	Performance	API responses must be returned within 2 seconds under normal load conditions.
NFR-02	Scalability	The database schema must support multiple institutes, courses, and hundreds of students.
NFR-03	Usability	The interface must be intuitive and accessible on both desktop and mobile browsers.
NFR-04	Reliability	Supabase managed PostgreSQL ensures 99.9% uptime with automated backups.
NFR-05	Maintainability	Codebase must follow modular structure with separate frontend, backend, and DB layers.

4.3 Hardware & Software Requirements

Development Environment

- Operating System: Windows 10/11 or macOS
- Node.js v18+ for backend runtime

Web Browser (Chrome / Firefox / etc.) for frontend testing
VS Code as the primary IDE
Git for version control

Deployment Environment

- Supabase Cloud for PostgreSQL database hosting
- Node.js server for backend API hosting
- Static file hosting for HTML/CSS/JS frontend

5. Entity-Relationship (ER) Diagram

The following diagram illustrates the six core entities of the VEI NEXUS database, their attributes, and the relationships between them. Primary Keys (PK) are highlighted and Foreign Keys (FK) are italicized.

5.1 Entity Descriptions

Entity	Description	Primary Key
ATTENDANCE_LOGS	Stores the attendance records of students across courses and dates	id
COURSE_POSTS	Stores announcements and resources for a course	id
ENROLLMENTS	Junction table linking students to the courses they are enrolled in	id
COURSES	Stores Course information	id
GRADE_LEVELS	Stores different types of grade levels	id
REGISTER_STUDENTS	Stores the registration form data of students	register_id
SIBLINGS	Stores siblings information	sibling_id
STUDENTS	Stores official record of accepted students	student_id
SUBJECTS	Stores Subjects information	Id
TEACHERS	Stores records of teachers	Id

5.2 Relationships

- GRADE_LEVELS (1) ----< REGISTER_STUDENTS(M): Many students can share common grade level
- GRADE_LEVELS (1) ----< SUBJECTS (M): Many Subjects can share same grade level
- REGISTER_STUDENTS (M) >----< REGISTER_STUDENTS(M) (through SIBLINGS): one student can have many siblings who are currently student in the institute
- REGISTER_STUDENTS (1) ---- STUDENTS (1): The record of student in register_students table is stored in STUDENTS table after acceptance
- STUDENTS (1) ----< ENROLLMENTS (M): A single students can be enrolled in many courses at a time

- STUDENTS (1) ----< ATTENDANCE_LOGS (M): A student can have multiple attendance records across different courses and dates.
- COURSES (1) ----< ENROLLMENTS (M): A single course can have many enrolled students
- COURSES (M) >---- SUBJECTS (1) : Many Instances of courses of a single subject can be taught at a time
- COURSES (1) ---< ATTENDANCE_LOGS (M) : A single course have attendance sheet of its students, since each attendance is considered as a record therefore a 1:M relationship forms
- COURSES (1) ---< COURSE_POSTS (M) : A single course will post many announcements and resources
- COURSES (M) >--- TEACHERS (1) : A teacher can teach many courses
- STUDENTS (M) >---< COURSES (M) (through ENROLLMENTS) : A student can take many courses and a course can have many students

5.3 Relationships / ERD

ER diagram File attached with the report

6. Normalized Database Schema

The VEI NEXUS database is designed in Third Normal Form (3NF). Each table satisfies the requirements of 1NF, 2NF, and 3NF as described below.

6.1 Normalization Overview

Normal Form	Rule	Status in VEI NEXUS
1NF	All attributes are atomic; no repeating groups	Satisfied — each column holds a single, indivisible value
2NF	No partial dependencies on composite primary keys	Satisfied — all tables use single-column surrogate PKs
3NF	No transitive dependencies between non-key attributes	Satisfied — each non-key attribute depends only on the PK
BCNF	Every determinant is a candidate key	Largely satisfied; minor deviations accepted for practicality

6.2 Table Schemas

Table 1: attendance_logs

Column	Data Type	Constraint	Description
id	UUID / SERIAL	PK NOT NULL	Unique identifier for each attendance record
student_roll_no	VARCHAR (20)	FK -> students NOT NULL	student roll number to record his/her attendance
course_id	INTEGER	FK -> courses NOT NULL	course id of course
attendance_date	DATE	NOT NULL	Date of attendance
status	VARCHAR (1)	CHECK IN ('P','A') NOT NULL	Status indicating absence or presence
(attendance_date, student_roll_no, course_id)	-	UNIQUE	Uniqueness constraint to prevent duplicates

Table 2: course_posts

Column	Data Type	Constraint	Description
id	UUID / SERIAL	PK NOT NULL	Unique identifier for each post
post_type	VARCHAR(20)	NOT NULL	Type of post. Initially there are only two types: Resource and Announcement
course_id	INTEGER	FK -> courses NOT NULL	course id of course
title	VARCHAR(150)	NOT NULL	Title/main heading of the post
content	TEXT	NOT NULL	Main content of the post
link	TEXT		Link if any posted
created_at	TIMESTAMP	DEFAULT NOW()	post creation timestamp

Table 3: courses

Column	Data Type	Constraint	Description
id	UUID / SERIAL	PK NOT NULL	Unique identifier for each course
subject_id	INTEGER	FK -> subjects NOT NULL	Subject id of the subject being taught in the course
teacher_id	INTEGER	FK -> teachers NOT NULL	Teacher id of teacher
is_published	BOOLEAN	DEFAULT FALSE	A state representing the live status of course
student_count	INTEGER	DEFAULT 0	Number of students currently enrolled in this course
created_at	TIMESTAMP	DEFAULT NOW()	Course creation timestamp

Table 4: enrollments

Column	Data Type	Constraint	Description
id	UUID / SERIAL	PK NOT NULL	Unique enrollment record ID
student_roll_no	VARCHAR(20)	FK -> students NOT NULL	Reference to the enrolled student
course_id	INTEGER	FK -> courses NOT NULL	Reference to the enrolled course
enrolled_at	TIMESTAMP	DEFAULT NOW()	Date and time of enrollment
(student_id, course_id)	—	UNIQUE	Prevents duplicate enrollments

Table 5: grade_levels

Column	Data Type	Constraint	Description
id	UUID / SERIAL	PK NOT NULL	Unique grade record ID
name	VARCHAR(50)	NOT NULL UNIQUE	Name of Grade level

Table 6: register_students

Column	Data Type	Constraint	Description
--------	-----------	------------	-------------

register_id	UUID / SERIAL	PK NOT NULL	Unique Registration record ID
full_name	VARCHAR (100)	NOT NULL	Full name of student
father_name	VARHAR (100)	NOT NULL	Father's name of student
guardian_contact	VARCHAR (50)	NOT NULL	Gaurdian's emergency contact number
date_of_birth	DATE	NOT NULL	Date of birth of student
date_of_registration	DATE	NOT NULL	Date of registration
grade_level_id	INTEGER	FK -> grade_levels UNIQUE	Grade level of student
class_year	VARCHAR(100)	NOT NULL	Class and year info of student
has_sibling	BOOLEAN	DEFAULT FALSE	A flag representing sibling count of student currently enrolled in institute
status	VARCHAR(30)	DEFAULT PENDING, CHECK IN ('PENDING','ACCEPTED','LEFT')	Current status of student

Table 7: siblings

Column	Data Type	Constraint	Description
sibling_id	UUID / SERIAL	PK NOT NULL FK -> register_students	Register id of sibling
register_id	INTEGER	PK FK -> register_students NOT NULL	Register id of student

Table 8: students

Column	Data Type	Constraint	Description
student_id	UUID / SERIAL	PK NOT NULL	Unique Student record ID
roll_no	VARCHAR(20)	NOT NULL	Roll number assigned by institute
password	VARCHAR(500)	NOT NULL	Password of student
accepted_at	TIMESTAMP	DEFAULT NOW()	Time recorded of application acceptance

Table 9: subjects

Column	Data Type	Constraint	Description
id	UUID / SERIAL	PK NOT NULL	Unique Subject record ID
name	VARCHAR(100)	NOT NULL	Name of subject
grade_level_id	INTEGER	FK -> grade_levels NOT NULL	Grade level of subject

Table 10: teachers

Column	Data Type	Constraint	Description
id	UUID / SERIAL	PK NOT NULL	Unique Teacher record ID
full_name	VARCHAR(100)	NOT NULL	Full name of teacher
password	TEXT	NOT NULL	Password of teacher account
phone	VARCHAR(50)	NOT NULL	Contact number of teacher
email	VARCHAR(50)	NOT NULL	Email of teacher
created_at	TIMESTAMP	NOT NULL	Timestamp at the time of registration

6.3 Foreign Key Relationships Summary

Foreign Key	From Table	References	On Delete
-------------	------------	------------	-----------

course_id	attendance_logs	courses(id)	CASCADE
student_roll_no	attendance_logs	students (roll_no)	CASCADE
course_id	course_posts	courses (id)	CASCADE
subject_id	courses	subjects (id)	SET NULL
teacher_id	courses	teachers (id)	SET NULL
course_id	enrollments	courses(id)	CASCADE
student_roll_no	enrollments	students (roll_no)	CASCADE
grade_level_id	register_students	users(user_id)	-
register_id	siblings	register_students (register_id)	CASCADE
student_id	students	register_students (register_id)	CASCADE
grade_level_id	subjects	grade_levels (id)	CASCADE

7. Conclusion

VEI NEXUS successfully demonstrates the design and implementation of a relational databasedriven web application for academic management. The system was built using a modern full-stack architecture — HTML/CSS/JavaScript on the frontend, Node.js on the backend, and Supabase (PostgreSQL) as the cloud-hosted relational database. The database schema is normalized to the Third Normal Form (3NF), ensuring data integrity, eliminating redundancy, and maintaining referential consistency through well-defined foreign key relationships.

The project covers all fundamental DBS concepts including entity-relationship modeling, schema normalization, constraint enforcement, and relational query design. The complete integration of all three tiers (presentation, business logic, and data) demonstrates a production-ready application architecture capable of supporting real institutional workflows.

This project was a collaborative effort by Mahad Khan (24K-0961), Shahbaz Hassan (24K-0884), and Samad Siddique (24K-1039), submitted in partial fulfillment of the Database Systems (DBS) course requirements for the Spring Semester 2024-25.